



Hadas AI

Running Routes Chat Bot

גיא דנין - 205372105

דוד ושלר - 209736578

רון אביטל - 318751419

```
Hadas: Welcome to the R&D route planner bot! I can help you plan a route for your next run. Let's get started!
You: Hi Hadas, generate a running route in Jerusalem for me
Hadas: I'm glad you're going to be there. What length feels comfortable for this route?
You: Lets run 15 kilometers today
Hadas: I'm glad you're going to run the next 15 kilometers. How challenging should the route be for you?
You: an easy route would me nice for me. and i want to start running from Yaffo street
Hadas: I'm glad you're going to go to the city. What is your desired endpoint?
You: i want to finish at Haneviim 37
Hadas: Alright! I'll plan a 15 km easy route starting at yaffo street and ending at haneviim 37.
Are you pleased with the route I've planned for you?
You: Ummm no
Hadas: Great! Let's keep going. Please provide the remaining details for your route.
You: I actually want to start from my location
Hadas: Greate! I'll build you a 15 km easy route from your location to haneviim 37.
Do you find the route I've planned for you satisfactory?
You: Yes thanks!
Hadas: I'm happy that you approve of the route I've planned for you. Enjoy your run!
```

מוגש לד"ר לרחל שפירא - עיבוד שפה טבעית המכללה האקדמית הדסה
ירושלים

תואר ראשון במדעי המחשב Bs.c

פברואר 2025

תוכן עניינים

2.....	תוכן עניינים
3.....	רקע
4.....	Data sets
5.....	Datasets - preprocessing
6.....	NLU - Deep learning attempt (failed)
8.....	NLU - MODEL CREATION PROCESS
10.....	TRACKER
11.....	POLICY
12.....	NLG
15.....	סיכום

רקע

במסגרת פרויקט גמר של רון אביטל ודוד ושלר העוסק באפליקציה ליצירת מסלולי ריצה, פיתחנו בוט שמעבד שפה כך שימלא את הפרמטרים המתאימים ליצירת מסלול ריצה מתוך שיחה עם משתמש. הפרמטרים שהבוט צריך למלא יהיו: מרחק, דרגת קושי, נקודת התחלה, נקודת סוף, אופי המסלול (מעגלי או A-B). הבוט מנהל שיחה אינטואיטיבית שמרגישה כמו שיחה עם חבר על מנת למלא את הפרמטרים על פי מה שלמדנו בקורס. בשלב הבא (בפרויקט הגמר) נשתמש בפרמטרים שהצלחנו להוציא באמצעות הבוט בתוך אלגוריתם ייעודי לבניית המסלול.

מדדי הצלחה:

- המודל ידע למלא פרמטרים הנוגעים למסלולי ריצה. הפרמטרים ישמשו את האלגוריתם שלנו בהמשך ליצירת המסלול.
- המודל חינמי לנו לשימוש.

תחומי אחריות:

מציאת ובניית datasets מתאימים - כולנו

NLU - רון

TRACKER - דוד

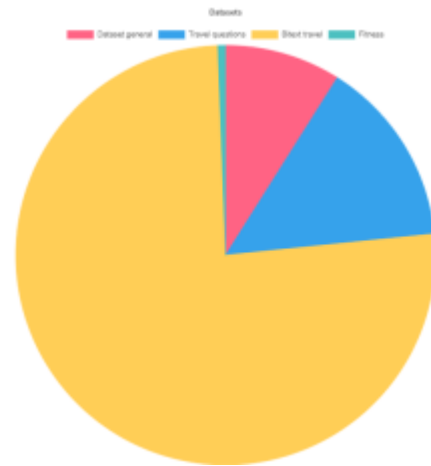
POLICY - דוד

NLG - גיא

Data sets

השתמשנו ב-4 דאטה סטס שונים בשביל מודל הלמידה עמוקה שלנו:

1. [Dataset for chatbot - kaggle](#) - מכיל דיאלוגים פשוטים שמתאימים לבניית צ'אטבוטים. כולל שני טורים: הראשון מכיל שאלות והשני תשובות. הסט כולל 3,725 פריטים.
2. [Travel question response - huggingface](#) - מכיל דיאלוגים של שאלות ותשובות בנושא טיולים. מכיל 3 עמודות: קלט, תגובה ראשונה ותגובה שניה. הסט כולל 6,009 פריטים.
3. [Bitext travel - huggingface](#) - הסט מיועד לאימון מודלים גדולים של שפה בתחום התיירות. הוא כולל 31,658 זוגות של שאלות ותשובות, עם כ-1,000 דוגמאות לכל כוונה. יש בו 33 כוונות שמחולקות ל-11 קטגוריות. בנוסף, הסט מכיל 10 תגי יצירת שפה.
4. [Fitness chatbot - huggingface](#) - דאטה סט שנועד לאמן צ'ט בוטים בתחום הכושר הגופני. מכיל כ-245 פריטים המורכבים משאלות ובקשות בתחום הכושר, תזונה, וכו'.



סכ"ה הגודל של הדאטה סט: 41,637 פריטים. בוצעו פעולות על הנתונים לפני הכנסתם למודל כדי להבטיח שהמודל יקבל קלט נקי, רלוונטי ומובנה היטב.

בכל גרסה של המודל תהליך העיבוד השתפר והפך ליותר ממוקד במסלולי ריצה.

1. ניקוי והכנה ראשונית של הנתונים
איחוד מבנה הנתונים – הפיכת כל הדאטאסטים לפורמט אחיד של "input output".
הסרת דוגמאות ריקות או בלתי קריאות – הוסרו תשובות חסרות.

2. סינון והגבלת הדאטאסטים לתחום הריצה
הוספת פילטרים חכמים – מפורט בהמשך.
התאמת תשובות קיימות – שינויים בתוכן התשובות כך שיהיו קשורות לריצה ולא לנושאים כלליים.

3. יצירת נתונים סינתטיים מותאמים אישית
הוספת 500 דוגמאות חדשות – עם שאלות ותשובות ספציפיות על מסלולי ריצה.
המרת מילים כלליות למונחים מעולם הריצה – לדוגמה, "trip" הוחלף ב-"running route".

לאחר הסינון קיבלנו דאטה סט בגודל 10234 לטובת אימון המודל.

Datasets - preprocessing

בשלב הראשון נטענו 3 מערכי נתונים ראשוניים מ-Hugging Face, עליהם פירטנו קודם, ואיחדנו אותם.

יתרון: קיבלנו מאגר גדול של דוגמאות שעוזר למודל ללמוד דפוסי שפה, ומכיל מגוון של ניסוחים טבעיים, כך שהמודל מקבל קלטים מציאותיים.

חסרון: הרבה מהשאלות והתגובות לא עסקו בריצה – למשל, המלצות על מלונות, מסעדות ותחבורה. בנוסף, שונות גבוהה בנתונים גרמה לכך שהמודל התקשה להבין הקשרים מדויקים. כמו כן לא היה סינון חזק של איכות התשובות, מה שפגע ברלוונטיות הפלטים.

בשלב השני הוספנו מערך נתונים נוסף: [Kaggle Dialogs](#), שגם עליו פירטנו קודם. בשלב זה בוצע סינון חזק יותר של הנתונים והותאמו תשובות מותאמות לריצה.

1. בחירה מחדש של הנתונים- נבחרו רק דוגמאות שעוסקות במסלולי ריצה מ Hugging Face.
2. התאמת תשובות כלליות לתחום הריצה על ידי הוספת מונחים שקשורים למסלולי ריצה כמו "trail", "flat path", "running distance".
3. הוספת נתונים סינתטיים

נוצרו 1300 דוגמאות חדשות עם קלטים מותאמים לריצה.

דוגמאות חדשות כללו בקשות מסוימות יותר כמו **"Find me a 5 km trail with slight elevation"**.

יתרון: המודל התחיל ללמוד ספציפית על ריצה, במקום לענות תשובות גנריות. הנתונים הפכו לעקביים יותר – כל שאלה עסקה במסלולי ריצה בלבד. הוספת מערך הנתונים החדש עזר לשיחה "החברית".

חסרון: עדיין היו בעיות בכתיבת תשובות טבעיות ונתונים הסינתטיים חזרו על עצמם יותר מדי.

בשלב השלישי, הדאטא סטים עברו אופטימיזציה מלאה לקראת האימון הסופי על מודל T5:

1. איחוד ושיפור של הנתונים הסינתטיים.
2. שיפור שיטת ההמרה של מונחים לא קשורים.
3. החלפנו מילות מפתח שמתארות טיולים במילים שמתאימות לשימושים שלנו. למשל את המילה "trip" החלפנו ב "route".

לאחר ההכנות על הדאטה, קיבלנו דאטה מספיק טוב כדי להתחיל לעבוד איתו במודל ה-NLG. במודל ה-NLU הזדקקנו למשהו שונה.

NLU - Deep learning attempt (failed)

ניסינו לעניין במגוון רב של מאגרי נתונים באינטרנט כדי למצוא משהו שיתקרב ככל הניתן לזה:

Create an **easy** running route from **my current location** to **jaffa 12** that is **6 km**.

I want to run **3.5 kilometers** starting at **hillel 5** and ending at **this location**, with **medium** difficulty.

Can you generate a **very hard** route from **king gorge 5** to **jaffa 62** with a distance of **7.8 km**?

לצערנו, הגענו למסקנה כי לא קיים מאגר נתונים כזה או משהו שדומה לכך. ולכן פנינו לבנות את המאגר של עצמנו.

הבסיס למאגר הנתונים שניסינו ליצור הסתמך על משפטי תבנית שיכילו בתוכם מקומות לסלולים שאותם נכניס במקומות הנכונים. כל סלוט יבחר באופן אקראי מתוך רשימה מתמאמה. כמו כן, נרצה לבצע bio-tagging לטובת המשך העיבוד וזיהוי ה-bio-tagging.

את הרשימת המיקומים והרחובות (המיקומים להתחלה וסוף המסלול) היינו צריכים להביא מתוך מאגר נתונים אחר שמכיל שמות של רחובות כאלה **בישראל**. על מנת שהצאנו שלנו ידע לזהות ביעילות שמות של רחובות כאלה. לצערנו, גילינו כי אין מאגרים חניניים כאלה. לכן השתמשנו בקלוד ai על מנת לייצר כמות גדולה של שמות רחובות ישראליים. במהרה גילינו כי קלוד מתרגם רחובות ישראלים בצורה שגויה. למשל את הרחוב 'כ"ט בנובמבר' שנמצא בכמה ערים בארץ, הוא תרגם ל: november 29. בעקבות זאת, המשכנו לחפש פתרונות ומצאנו אתר בשם [geographic](#) שמכיל דפים עם רחובות ישראלים, ובנינו סקריפט שעושה data scarping על מנת ליצור קובץ עם כ-13,000 רחובות בארץ.

לאחר שיצרנו את רשימת התבניות, רשימת דרגות הקושי, רשימת המיקומים (של התחלה וסוף המסלול- אותה הרשימה), רשימת אורכי המסלולים האפשריים (יכולנו להשתמש בטוקן ספרה אבל רצינו לבצע preprocessing בשלב נפרד כי היו עוד כמה דברים לטפל בהם, עוד על כך בהמשך), כעת יכולנו לייצר את מאגר הנתונים. אבל נתקלנו בבעיה. בגלל שביצענו את ה-bio-tagging שלנו באמצעות **bert** ו-**NER**, ובגלל שהפכנו את שמות הרחובות שלנו להיות עם אותיות קטנות בלבד (מתוך הנחה שמשמש אמיתי לא יטרח להבדיל בין אותיות קטנות וגדולות בכתיבה שלו כלל) גילינו כי ה-bio-tagging שלנו אינו מוצלח כלל. [ניתן לראות יותר בקובץ זה](#).

לכן, החלטנו לשנות גישה. הבנו שאין צורך ב-bert ו-NER כי כל המידע כבר נמצא מולנו בתבניות שהכנו. ה-bio tags כבר נמצאים שם ולכן אין צורך להשתמש בכלים כאלה. במקום זאת, השתמשנו ברגקסים וזיהוי התבנית שנמצאת בתוך ה-{}. דבר זה שיפר דרמטית. [ניתן לראות יותר בקובץ זה](#). [לעיון בקוד שבו ערכנו את כל הניסיונות השונים](#).

כעת, חשבנו להגדיל את כמות התבניות למשפטים האפשריים כך שמודל ה-NLU יוכל להתאמן גם על משפטים יותר "עממיים" / "חבריים". לשם כך ניסינו לבצע data scrapping מ-reddit בסאב-רדיטים שקשורים לעולם הריצה. מחיפוש ידני לא מצאנו אף בקשה למסלול שדומה למה שאנחנו צריכים ולכן ניסינו לחפש בכמה שיותר סאב-רדיטים מילות מפתח שמעידות שפוסט זה מבקשים הצעה למסלול ריצה. הרצנו סקריפט שעבר על 7 סאברדיטים בנושאי ריצות וכושר, בכל

אחד מהם קראנו את 500 הפוסטים האחרונים ואת כל התגובות שבהם. קיבלנו 178 תוצאות. לאחר התעמקות בהם, אף לא אחד מהם באמת מכיל בקשה ליצור מסלול (אלא סתם את מילות המפתח). נצרך את [הקוד שעושה זאת \(ללא מפתח ה-API\) כאן](#). במקום זאת, יצרנו באופן סינטי באמצעות AI משפטי בקשות "עממיים" / "חבריים" למסלולי ריצה על פי הסלוטים שלנו. הוא הפיק במצבטבר 444 דוגמאות כאלה. כעת אנו מוכנים לייצא את מאגר הנתונים שלנו. [ניתן להתבונן בהם כאן](#).

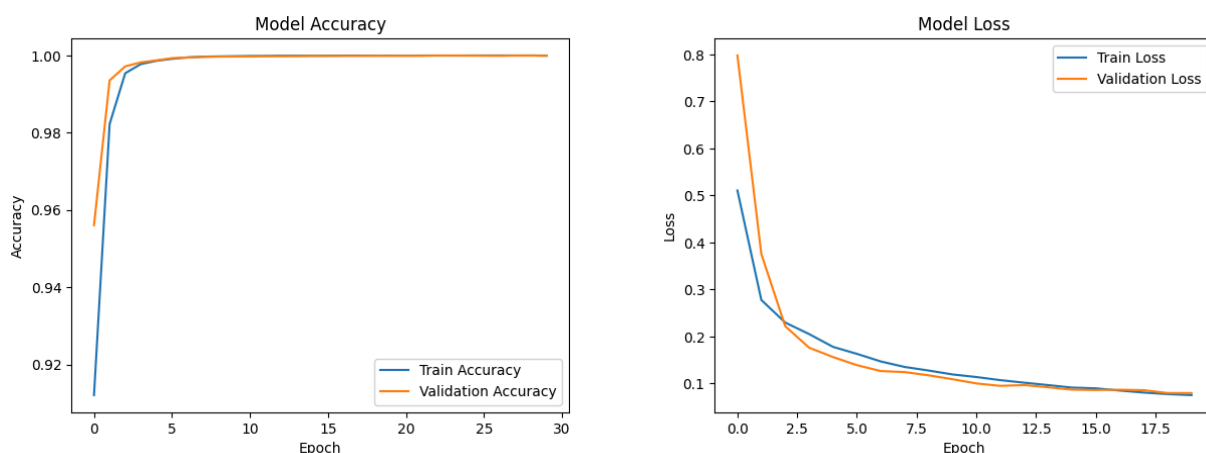
לסיכום: מבנה נתונים זה מתבסס על: 499 תבניות, 14 רמות קושי שונות, 153 אורכי מסלולים שונים, 944 מיקומים. יצרנו מספר גרסאות בנפחים שונים של מאגר הנתונים. מאגר נתונים זה יעזור לנו לאמן באמצעות *deep learning supervised method* את מודל ה-NLU שלנו.

כאמור יצרנו 499 תבניות בקשה שונות. עם זאת, מדובר בגודל מאוד קטן כשאנו באים לעסוק בעולם של NLP. לכן, כאשר יצרנו מאגרי נתונים עם אלפי דוגמאות קל להבין כי תבניות רבות מוכפלות מספר פעמים עם פרמטרים שונים. למעשה, אין הבדל בין דוגמא מסוימת שמבקשת מסלול למיקום א' לאותה דוגמא שמבקשת מסלול למיקום ב'. לכן, בעת אימון המודל קיבלנו overfitting מכיוון ש-test ו-train מעורבבים. לצערנו, בנקודה זו הבנו כי לא נוכל לאמן את מודל ה-NLU ב-deep learning משום שאין לנו מאגר נתונים מספק. פנינו לבחון אפשרויות לזיהוי bio taggings בשיטות של rules based ושימוש במודלים קיימים. [המודל זמין לצפייה כאן](#).

תיאור מפורט של מבנה המודל שבנינו:

1. **Input Layer**: מקבל את הקלט.
2. **Embedding Layer**: מפה כל מילה (או טוקן) ל vector באורך 100.
3. **Bidirectional LSTM Layer**
4. **Dropout Layer**: על מנת לנסות למנוע overfitting, עם ערך 0.2.
5. **BatchNormalization Layer**: לנרמל את הפלט.
6. **TimeDistributed Dense Layer**: משמש למיון טוקנים עם פונקציית הפעלה softmax.

המודל מוכלל עם אופטימיזר Adam.



מובן כי לא ניתן להסתמך על העקומות שקיבלנו בעקבות מה שתיארנו לעיל אך צירפנו אותם כי דרכם הצלחנו להגיע למסקנות הללו.

NLU - MODEL CREATION PROCESS

בפרק זה נתאר את מהלך בניית המודל שמזהה את bio tagging הדרושים עבור הצאת בוט. באופן ספציפי, נתמקד באיתור שמות המיקומים עבור התחלה וסוף מסלול הריצה שכן כאן היה האתגר העיקרי. נגדיש כי מדובר בשני דברים שונים. האתגר העיקרי באיתור שמות מיקומים היא שבישראל שמות רחובות מתורגמים באופן ישיר משמיעה לאותיות ואין להם מספר תבניות מוגבל בו הם עשויים להופיע, במיוחד בכתיבה חופשית של משתמש. קושי נוסף הוא הבחנה בין מיקום התחלה למיקום סוף. למשל, הישענות על המילה "at" לפני שם המיקום לא מספקת מידע האם מדובר במיקום התחלה או סוף. קושי נוסף הוא עיסוק עם מילים רבות משמעות כמו "to" שמשמשת גם עבור purpose וגם למיקום.

לאחר מחקר מעמיק בנושא כיצד כדאי לגשת לבעיה זו מצאנו כי כדאי להשתמש במודלים קיימים של NER בשילוב של מערכת חוקים. לשמחתנו, גילינו כי ישנן לא מעט אפשרויות בתחום זה. בכל הדוגמאות הבאות הרצנו את הדוגמאות על המשפטים:
"I live on 123 Main Street and work near 5th Avenue" (לצורך ביקורת)
"I need a route from haneviim 37" (משפט פשוט שצריך להצליח)
עשינו זאת מתוך מחשבה שאם השיטות יוכלו לזהות שמות של רחובות במצבים הללו נוכל להמשיך ולחקור עוד עליהם. [ניתן למצוא את המחברת שמנסה את הגישות השונות כאן.](#)

• spaCy with Pretrained Models

ניסינו לעבוד עם 3 מודלים מתוך spaCy:
`en_core_web_sm`, `en_core_web_md`, `en_core_web_lg`
השוני העיקרי ביניהם הוא בגודל שלהם. ניסינו להריץ את המודלים הללו עם התגיות "LOC", "FAC", "GPE" על מנת לכוון את החיפוש למיקומים באופנים שונים.
תוצאה: רשימה ריקה. לא נמצאו תוצאות.

• Hugging Face Transformers

השתמשנו ב- `dbmdz/bert-large-cased-finetuned-conll03-english` גם לא קיבלנו הצלחה. מתוך כל המילים הוא זיהה רק חלק מהמילה שמייצגת את הרחוב וגם בזה הוא נתן תיוג מוזר.

• Flair

השתמשנו במודל "flair/ner-english" וקיבלנו גם כאן, כלום, אובייקט ריק.

• Stanford NLP - Stanza

קיבלנו: 37 CARDINAL
שלמעשה אומר שהוא זיהה את המספר 37 ככמות/ כמספר של משהו. זה אמנם יותר מכל מה שראינו עד כה אבל זה עדיין כלום.

אם כך, הסתפקנו במודל הבסיסי והמינימלי ביותר שעבד על דוגמאות של רחובות ומיקומים אמריקאים en_core_web_sm רק על מנת שיהיה לנו את האפשרות לתפוס בתיוגים גם את המקרים הללו. פנינו לבניית המודל על סמך חוקים קבועים מראש.

לאחר מאמצים, יצרנו מודל שיועד למלא את ה-bio tags השונים שלנו בהצלחה. הארכיטקטורה הייתה יצירת "פונקציית אב" שתדע לקבל את הפוקוס מ-policy ובכך לתעדף חיפוש ב-"פונקציה מומחית" עבור הפוקוס הנתון. בדרך זו, המודל ידאג תחילה למלא את ה-bio tagging ש-policy ביקש ממנו קודם ולאחר מכן יבדוק האם ישנם דברים נוספים שניתן למלא. גילנו כי שיטה זו עובדת באופן הטוב ביותר. [ניתן לראות את המחברת של המודל \(באופן בלתי תלוי, ללא חיבור לשאר המודלים\) כאן.](#)

בהינתן כמות גדולה יותר של משאבים היינו רוכשים את הרשימה של הרחובות בישראל ומשתמשים בה על מנת לזהות מיקומים, בנוסף, היינו אוספים מהמשתמשים שלנו את השיחות שלהם עם הצאט כך שנוכל לאמן מודל ב-deep learning ובכך להחליף בסופו של דבר את המודל הנוכחי מתוך הבנה שהוא יוכל להיות טוב מאוד אם יהיו לו מספיק נתונים.

TRACKER

ה-tracker שלנו ממומש באמצעות מחלקה, עם member של אובייקט של סלוטים. המחשבה ההתחלתית הייתה, שממודל ה-NLU נקבל את ה-bio tagging של המשפט מהמשתמש, ונמצא את הסלוטים החסרים עד למילוי כל הסלוטים.

תוך כדי עבודה הבנו שאנו צריכים לבצע 2 שינויים עיקריים:

- (1) **סלוט שכבר מולא יכול להתעדכן**, מתוך הבנה שהמשתמש יכול לשנות את דעתו תוך כדי השיחה, או לחילופין המודל שלנו יכול לזהות באופן שגוי את אחד הסלוטים, ולאחר מכן המשתמש ירצה לתקן אותו.
- (2) **2 מתוך 6 הסלוטים שלנו לא תמיד הכרחיים** (מספרי הרחובות של נקודת ההתחלה ונקודת הסוף). משתמש לפעמים ירצה להתחיל לרוץ מהמיקום הנוכחי שלו, מרחוב כלשהו ללא מספרים, או שפשוט לא יהיה אכפת לו מאיזה מספר ברחוב הוא רוצה להתחיל/לסיים. לכן קילבנו החלטה ששני הסלוטים האלה לא יהיו חסם לסיום השיחה.

לאחר ביצוע השינויים האלו, התחלנו לקבל תוצאות יותר יפות.

בעיה נוספת שנתקלנו בה היא שסדר המילים שהגיע מה NLU ל tracker השתנה, כך שקיבלנו שמיקום ההתחלה יכל להיות george king במקום king george. בעיה זו הובילה לפלט הבט מהבט:

"I will create a 8 km easy route from George King street to Yafo 4"

כדי להתגבר על הבעיה הזאת, שינינו את סדר הכנסת המילים המתוייגות, כך שיכניס קודם תיוגים שמתחילים ב B ורק אז את האלו שמתחילות ב I. שינוי זה פתר לנו את הבעיה, ולאחר מכן קיבלנו את הפלט המתוקן:

"I will create a 8 km easy route from King George street to Yafo 4"

POLICY

ה policy שלנו גם ממומש באמצעות מחלקה ופונקציית main. התחלנו במימוש פשוט שלו - הוא יקבל מה tracker את רשימת הסלטים הלא מלאים, ממיינים על ידי סדר החשיבות שלהם, ועל בסיס הסלוט החשוב הבא, הוא יקבל החלטה על אופי השאלה הבאה שהבוט ישאל את המשתמש.

מימוש זה קיבל שדרוג - את הסלוט שבו אנו מתרכזים אנו שולחים למודל ה NLU, על מנת שיוכל לנתח בצורה טובה יותר את המשפט מהמשתמש.

בנוסף, הוספנו ל policy מורכבות - כאשר השיחה מתחילה להתארך (קובעים זאת על ידי פרמטר שעוקב אחרי מספר ההודעות שנשלחו הלוך חזור מהמשתמש), ה policy מקבל החלטה לשאול את המשתמש אם הוא רוצה להמשיך בשיחה או לסיים ולקבל מסלול על סמך הפרמטרים שכבר מצאנו. זאת על מנת למנוע שיחה ארוכה מידי, שכן השיחה נועדה להיות קצרה ומדויקת, בניגוד לצ'אט בוטים אחרים שנועדו לשם שיחה זורמת וארוכה.

דבר נוסף ששמנו לב שחסר הוא וידוי הפרטים מול המשתמש בסיום השיחה. בעקבות זאת הוספנו החלטה - כשאר כל הסלטים מלאים והבוט מוכן לסיים, הוא יסכם למשתמש את הפרטים שהוא אסף, וישאל האם המשתמש מרוצה מהתוצאה. אם כן - נשלח הודעת סיום ונסיים. אם לא - נבקש מהמשתמש פרטים נוספים.

כאן ניתן לראות את הבוט מאמת את הפרטים מול המשתמש:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE COMMENTS
PS C:\Users\שמתשם\OneDrive\דיוקנה\CS_new\third year semester 1\NLP\project\NLP_ChatBot\NLP_ChatBot\fullstack> python main.py
Hadas: Greetings! I'm the R&D route planner bot. I'm here to help you plan a route for your run. Let's start!
You: Hi hadas, help me plan a nice route for this sunny day
Hadas: i'm glad you're going to be there for a sunny day. How many kilometers do you have in mind for the route?
You: the route should be around 10 kilometers long
Hadas: i'm glad you're going to be there for a long time. What level of challenge suits you best?
You: a very advanced trail is good for me
Hadas: i'm glad you're going to be there. What is your preferred start point?
You: lets start from my current location, and end at Yaffo 18
Hadas: Alright! I'll plan a 10 km very advanced route starting at your current location and ending at yaffo 18.
Are you pleased with the route I've planned for you?
You: not really
Hadas: Got it! Let's keep moving forward. Please provide the remaining details for your route.
You: i actually want to run 5 kilometers, fix that
Hadas: Greate! I'll build you a 5 km very advanced route from your current location to yaffo 18.
Are you happy with the route I've planned for you?
You: yes maam
Hadas: Excellent! I'm glad you're content with the route I've planned for you. Have a great run!
PS C:\Users\שמתשם\OneDrive\דיוקנה\CS_new\third year semester 1\NLP\project\NLP_ChatBot\NLP_ChatBot\fullstack> |
```

קושי במודל זה: לאחר ששואלים את המשתמש אם הוא מרוצה מהתוצאות, במידה והמשתמש אומר שלא - אנחנו מבקשים ממנו לתת עוד פרטים אבל מעתה והלאה אין סלוט שבו אנחנו מתרכזים. ניתן לשפר את המודל על ידי ניתוח התשובה מהמשתמש על מנת להבין מה הוא רוצה לשנות. אבל - על מנת לעשות זאת נצטרך להרחיב בהרבה את הפונקציות של ה rules או לחילופין - לאמן מודל שיזהה את הסלטים שאותם המשתמש רוצה לשנות (מה שאין ביכולתנו לעשות ללא דאטה מספיק גדול).

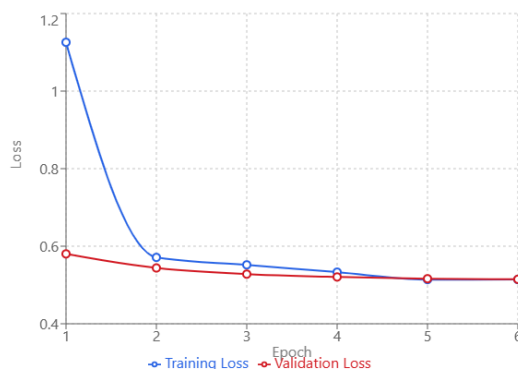
NLG

בניית המודל כללה ניסיונות רבים, כאשר בכל ניסיון שכללנו את אופן עיבוד הנתונים (מתואר בפרק preprocessing). נתאר את המודלים שנבחרו, את השינויים שבוצעו ואת התוצאות שהמודל הפיק.

DistilGPT-2

מודל זה הינו מודל גנרטיבי שמתבסס על GPT-2, אך קטן יותר וקל יותר לאימון. ביצענו את ה-fine tuning הבא:

- learning_rate = 2e-5
- batch_size = 8
- num_train_epochs = 6
- weight_decay = 0.01



Epoch	Training Loss	Validation Loss
1	1.125800	0.580162
2	0.571100	0.543922
3	0.551900	0.527962
4	0.533100	0.520663
5	0.513900	0.516159
6	0.514000	0.514654

תוצאה: זיהינו למידה יציבה (בין סט האימון למבחן) אך בפועל התוצאות היו לא טובות ולכן הוחלט לנסות מודל אחר.

דוגמה של תשובה מהמודל:

Generated Response: Plan a route for me from my home to work. Here, I'd like to suggest a scenic route for you: the Appalachian Trail, which runs from Annapolis to the Virginia Tech USA, is a great option for me. I'd be happy

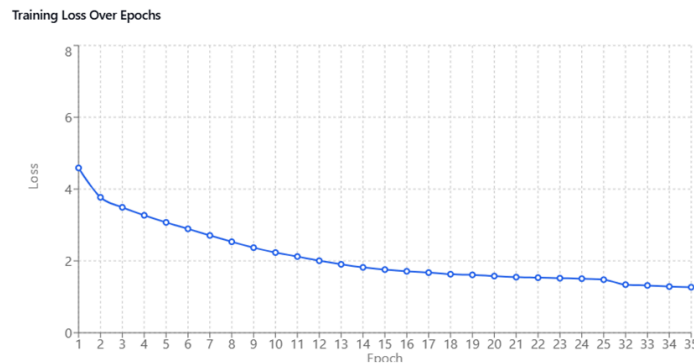
Seq2Seq with LSTM + Attention

במקום להשתמש במודלים מוכנים כמו GPT-2, כאן נבנה מודל ייעודי שמבוסס על רשת LSTM עם מנגנון Attention. מבנה המודל:

1. Embedding Layer הופך את הטקסט למספרים (וקטורים) שניתן להזין לרשת הניורנית.
2. Encoder – LSTM מקבל את הקלט, מקודד אותו ומייצר ייצוג פנימי.
3. Decoder – LSTM מקבל את הייצוג הפנימי מה-Encoder ומייצר רצף מילים חדש.
4. Attention Mechanism מאפשר ל-Decoder להתמקד על חלקים חשובים של הקלט כדי לשפר את הדיוק.
5. Fully Connected Layer בשכבה האחרונה מחזיר לוגיטים שמתורגמים חזרה למילים.

למה בחרנו ב-Seq2Seq עם Attention?

- טוב במיוחד למשימות גנרציה של טקסט, כגון יצירת תשובות מותאמות.
- טוב במיוחד למשימות גנרציה של טקסט, כגון יצירת תשובות מותאמות.
- מנגנון ה-Attention עוזר למודל להבין אילו מילים חשובות יותר.
- מאפשר שליטה על התהליך הגנרטיבי, בניגוד ל-GPT-2 שנותן טקסטים חופשיים יותר.



גם כאן הלמידה הייתה יציבה אבל שוב התוצאות בפועל לא היו מספקות. ולכן ניסינו מודל נוסף.

דוגמה של תשובה מהמודל:

You: i want to plan running route

Model: that sounds like fun

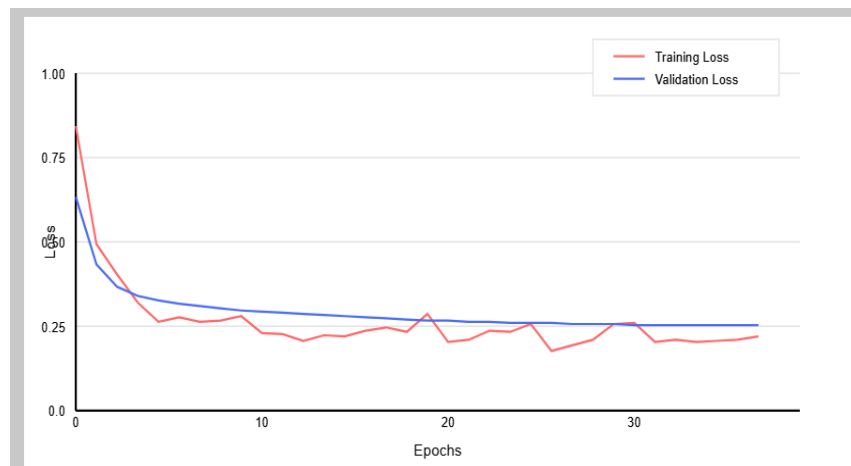
You: help me plan it, i want 5 km route

Model: we need to save enough

Fine-tuned T5

בשלב זה הוחלט לעבור ממודל מבוסס LSTM למודל מבוסס טרנספורמרים - T5 מודל רב-תכליתי שמותאם למשימות מבוססות טקסט (כולל תרגום, סיכום, ושאלות-תשובות). תומך בשיטת למידה גנרטיבית, כך שהוא מתאים במיוחד להפקת תשובות מותאמות אישית. נוח לביצוע Fine-Tuning על משימה חדשה, במיוחד עם דוגמאות ממוקדות כמו כאן. בנוסף, עשינו טוקניזציה משופרת, נוספו מונחים רלוונטיים לריצה ללקסיקון של המודל כדי לשפר את איכות ההפקה. אסטרטגיית הפקה משופרת (Decoding Strategy) שימוש ב-Beam Search במקום Greedy Decoding כדי לייצר תשובות מדויקות יותר. יתרונות של המעבר ל-T5 בהשוואה למודל הקודם:

- שיפור משמעותי בדיוק התשובות - מודל מבוסס טרנספורמרים מסוגל להבין הקשרים מורכבים יותר מאשר LSTM.
- מהירות אימון גבוהה יותר - T5 משתמש בפרלליזציה ולכן האימון מהיר יותר.
- גמישות רבה יותר בשיפור התשובות - ניתן לשלוט על אורך ותוכן התשובות באמצעות Beam Search.



דוגמה של תשובה מהמודל:

Input: How are you?

Model Response: i'm not sure.

Input: Find me a 10 km trail near my current location.

Model Response: Looking for a trail? Let me find you a running route in this area!

Input: I am looking for a flat 3 km route near my home.

Model Response: I'd be happy to help you find a flat 3 km route near my home.

Input: Can you suggest a scenic route for 5 km?

Model Response: Absolutely! I'd be happy to help you plan a scenic route for 5 km. Here are some suggestions:

מבין שלושת המודלים, המודל השלישי הניב את התוצאות הטובות ביותר המבחן התוצאהף ולכן בחרנו להמשיך איתו בבניית הבוט.

סיכום

באופן כללי, אנו יחסית מרוצים מהתוצר של הצאט שיצרנו. הוא יודע להגיב ולמלא את הערכים המתאימים בשיחה שאינה מתחכמת ויודע לנהל שיחה באופן זורם וחלק. עם זאת, אנו חושבים שעם יותר משאבים ליצירה של דאטא משל עצמנו (למשל על ידי איסוף ממשתמשים בבוט או השקעה של סכום כסף באנשים שיצרו את הדאטא על ידי שיחות) היינו יכולים לייצר מודל ב deep learning גם עבור ה-NLU. בנוסף, היינו מעוניינים לרכוש את המאגר של הרחובות בישראל מתוך הבנה שזה יוכל לתרום גם לצאט בהבנת המיקומים (טרם שילוב deep learning) והן בפיתוח פרויקט הגמר (בקריאות API. לא ניכנס לכך כעת). עבור מודל ה-NLG בהינתן יותר כוח חישובי לדעתנו יכולנו להגיע לתוצאות יותר מרשימות, הגדלת כמות הדאטא לטובת אימון המודל הייתה מאפשרת ליצר מודל יותר כללי ובכך היינו מסוגלים לגוון את הדרכים שבה ה-NLU עונה. באופן אידיאלי, היינו מעוניינים לחשוב על דרך שבה מודל ה-NLG גם מסוגל לשאול את השאלה הבאה באמצעות פרמטר של נושא בלבד מה-POLICY (בניגוד למה שקורה כרגע, ש-POLICY מעביר את השאלה כולה).

הוראות הרצה:

הקוד מוגש בקובץ זיפ יחד עם קובץ זה. ניתן להוריד אותן ולהריץ אותו על ידי ההוראות שנפרט כאן, או לחילופין להוריד אותו [מגיטהב בקישור הזה](#), ולפעול על פי ההוראות ברידמי.

1. יש להוריד את התוכנית ולפתוח אותו בסביבת עבודה.
2. יש לתקין את הספריות הדרושות בעזרת הפקודה: `pip install -r req.txt` (ייתכן שצריך להתקין ספריות נוספות).
3. יש להוריד את מודל ה-NLG מהדרייב [בקישור הזה](#), לשמור אותו לוקאלית במחשב והעתיק את הנתבי לתיקייה.
4. יש להוריד את מודל ה Spacy בעזרת הפקודה:
`python -m spacy download en_core_web_sm`
5. כעת ניתן להריץ את הבוט בעזרת הפקודה:
6. `python main.py <path to downloaded folder>` (לא לשכוח להחליף את `path_to_download_folder` בנתיב שהעתקנו קודם.